Kasi Yedumati · Full Stack Training

Aggregations and Group By

Module Outcome:

- Turn many order rows into COUNT / SUM / AVG a manager can use
- GROUP BY category or brand without “invalid in select list” panic
- Filter groups with HAVING — not WHERE on the aggregate

Note: This chapter uses **MySQL** syntax. String literals use single quotes (`'value'`).

We perform aggregations in such scenarios to combine multiple values into a single value, i.e., individual scores to an average score.

Aggregations

Combining multiple values into a single value is called aggregation. Following are the functions provided by SQL to perform aggregations on the given data.

Aggregation Function	Description
COUNT	Count the number of values
SUM	Adds all the values
MIN	Returns the minimum value
MAX	Returns the maximum value
AVG	Calculate the average of the values

Syntax:

```
SELECT
    aggregate_function(C1),
    aggregate_function(C2)
FROM
    TABLE;
```

Note: We can calculate multiple aggregate functions in a single query.

Examples:

1. Get the total runs scored by 'Ram' from the player_match_details table.

```
SELECT
    SUM(score)
FROM
    player_match_details
WHERE
    name = 'Ram';
```

Output: SUM(score) = 221

2. Get the highest and least scores among all the matches that happened in the year 2011.

```
SELECT
    MAX(score),
    MIN(score)
FROM
    player_match_details
WHERE
    year = 2011;
```

Output: MAX(score) = 75, MIN(score) = 62

COUNT Variants

Calculate the total number of matches played in the tournament.

Variant 1:

```
SELECT COUNT(*)
FROM player_match_details;
```

Variant 2:

```
SELECT COUNT(1)
FROM player_match_details;
```

All variants give the same result: **18**.

Note: In MySQL, use `COUNT(*)` or `COUNT(1)`. `COUNT()` without an argument is **not valid** in MySQL.

Note: In SQL, there's a difference between using COUNT(*) and COUNT(column_name):

- `COUNT(*)`: Counts the total number of rows in a table, regardless of whether any specific column contains NULL values.
- `COUNT(column_name)`: Counts the number of Non-NULL values in the specified column.

Special Cases

- When `SUM` function is applied on non-numeric data types like strings, date, time, datetime, etc., MySQL returns `0`.

Aggregate Functions on Strings	Output
MIN, MAX	Based on lexicographic ordering
SUM, AVG	0
COUNT	Default behaviour

- NULL values are ignored while computing the aggregation values.

Aggregate Functions on only NULL values	Output
MIN	NULL
MAX	NULL
SUM	NULL
COUNT	0
AVG	NULL

Alias

Using keyword `AS`, we can provide alternate temporary names to the columns in the output.

Syntax:

```
SELECT
    C1 AS a1,
    C2 AS a2,
    ...
FROM
    table_name;
```

Example: Get all the names of players with column name as 'player_name'.

```
SELECT
    name AS player_name
FROM
    player_match_details;
```

Output:

player_name
Ram
Joseph
...

Group By

The `GROUP BY` clause in SQL is used to group rows which have same values for the mentioned attributes.

Syntax:

```
SELECT
    c1,
    aggregate_function(c2)
FROM
```

```
    table_name
GROUP BY
    c1;
```

Example: Get the total score of each player in the database.

```
SELECT
    name, SUM(score) AS total_score
FROM
    player_match_details
GROUP BY
    name;
```

Output:

name	total_score
David	105
Joseph	116
Lokesh	186
...	...

GROUP BY with WHERE

We can use WHERE clause to filter the data before performing aggregation.

Syntax:

```
SELECT
    c1,
    aggregate_function(c2)
FROM
    table_name
WHERE
    c3 = v1
GROUP BY
    c1;
```

Example: Get the number of half_centuries scored by each player.

```
SELECT
    name, COUNT(*) AS half_centuries
FROM
    player_match_details
WHERE
    score >= 50
GROUP BY
    name;
```

Output:

name	half_centuries
David	1
Joseph	2
Lokesh	3
...	...

HAVING

HAVING clause is used to filter the resultant rows after the application of GROUP BY clause.

Syntax:

```
SELECT
    C1,
    C2,
    aggregate_function(C1)
FROM
    table_name
GROUP BY
    C1, C2
HAVING
    condition;
```

Example: Get the name and number of half-centuries of players who scored more than one half century.

```
SELECT
    name,
    COUNT(*) AS half_centuries
FROM
    player_match_details
WHERE
    score >= 50
GROUP BY
    name
HAVING
    half_centuries > 1;
```

Output:

name	half_centuries
Lokesh	2
Ram	3

Note: WHERE vs HAVING

- WHERE is used to filter rows and this operation is performed before grouping.

- HAVING is used to filter groups and this operation is performed after grouping.

> **Note:** In MySQL, `HAVING half_centuries > 1` using a column alias works. For strict SQL standard compliance, repeat the expression: `HAVING COUNT(*) > 1`.